

Erlind Sejdiu

106357

**Exposé: Evaluierung des nRF-Ökosystems und
Zephyr RTOS für ressourcenbeschränkte
IoT-Anwendungen anhand eines
Wearable-Prototyps**

**Projektarbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Sommersemester 2026**

Korrektor: Prof. Dr. rer. nat. Carsten Sinz

Inhaltsverzeichnis

1	Motivation	1
2	Problemstellung	1
3	Stand der Technik	2
4	Forschungsfrage	3
5	Methodik	3
6	Zeitplan	5
	Literaturverzeichnis	7

1 Motivation

Durch die fortschreitende Digitalisierung und zunehmende Vernetzung werden intelligente, eingebettete Systeme zunehmend Teil unseres Alltags. Selbst kleinste Alltagsgegenstände sind heutzutage in der Lage mit ihrem Umfeld zu kommunizieren und Daten zu erfassen. Besonders deutlich wird diese Entwicklung im Bereich von Smart-Home-Technologien und tragbaren Geräten, sogenannten Smart Wearables. So entwickelt beispielsweise das Unternehmen DIO intelligente Ringe, die biometrische Daten wie den Puls sowie Bewegungsparameter erfassen können [1]. Auch moderne Smart-Home-Systeme ermöglichen eine kontinuierliche Überwachung von Umgebungsbedingungen, etwa der Luftqualität, und übertragen diese Daten zur Analyse an mobile Endgeräte [2].

In vielen dieser Smart-Devices haben sich die nRF-System-on-Chips (SoCs) von Nordic Semiconductor etabliert, da sie hinsichtlich Energieeffizienz, Baugröße und Funktionalität gängige Industriestandards erfüllen. Die Nachteile dieser leistungsfähigen Hardware ist jedoch ein hochkomplexes Software-Ökosystem. Für moderne nRF-Chips wird offiziell die Nutzung des Echtzeitbetriebssystems Zephyr RTOS in Verbindung mit dem nRF Connect SDK empfohlen. Diese Toolchain führt jedoch abstrakte Konzepte wie DeviceTrees und Kconfig ein, denen eine steile Lernkurve zugesprochen wird [3], [4].

2 Problemstellung

Das zentrale Problem ist die Unklarheit zwischen dem hohen Konfigurationsaufwand des Zephyr RTOS und dem daraus resultierenden funktionalen Mehrwert bei der Programmierung moderner SoCs. Obwohl Zephyr zunehmend als Industriestandard empfohlen wird, gilt die Entwicklung als komplex, da spezifische Konzepte wie DeviceTrees und Kconfig ein tiefgehendes Systemverständnis voraussetzen [4].

Zur klaren Abgrenzung der Problemstellung liegt der Fokus der Arbeit ausschließlich auf der Evaluation des Aufwandes der Embedded-Softwareentwicklung und der Architektur, nicht auf dem Hardware- oder PCB-Design. Ebenso ist es nicht das Ziel valide medizinische oder industrielle Testdaten zu erheben. Das Hauptaugenmerk liegt vielmehr auf der softwareseitigen Evaluierung und der Komplexität, die zur Implementierung der jeweiligen Features bewältigt werden muss hinsichtlich Zephyr und den nRF Nordic Semiconductor SoCs.

3 Stand der Technik

In der aktuellen Literatur finden sich bereits diverse Vergleiche von Betriebssystemen für eingebettete Systeme bezüglich Zephyr. Meistens liegt der Fokus dabei jedoch auf allgemeinen Architekturkonzepten oder reinen Leistungsmessungen, während die praktische Entwicklerperspektive nicht berücksichtigt wird.

Ein Beispiel für einen solchen theoretischen Vergleich liefert die Arbeit von Kanwal et al. [5]. Die Autoren evaluieren verschiedene IoT-Betriebssysteme anhand ihrer Architektur, Programmiermodelle, Scheduling-Verfahren und Speicherverwaltung. Diese Analyse verbleibt jedoch auf einer rein konzeptionellen Ebene und trifft keine verlässliche Aussage darüber, wie aufwendig sich die praktische Konfiguration und Implementierung in den jeweiligen Systemen gestaltet.

Andere Arbeiten dringen zwar tiefer in die Praxis ein und testen Systeme wie Zephyr, RIOT und Contiki-NG auf ressourcenbeschränkter Hardware, jedoch beschränken sich auch diese Benchmarks primär auf die reine Netzwerkleistung. Der tatsächliche Entwicklungsaufwand für die Schaffung dieser Programmfunktionalitäten wird nicht bewertet [6].

Neben wissenschaftlichen Publikationen existieren zahlreiche praxisnahe Artikel aus der Industrie, wie beispielsweise der Beitrag „Measuring Real-Time Operating System Performance Part II: Comparing FreeRTOS vs. Zephyr“ [7]. Derartige Analysen konzentrieren sich oft eng auf hardwarenahe Laufzeitmetriken wie Taktzyklen, Interrupt-Latenzen oder die Dauer eines Context-Switches.

Es lässt sich festhalten, dass der aktuelle Stand der Technik vorrangig die Hardware-Performance oder die theoretische Architektur der Betriebssysteme analysiert. Was jedoch weitreichend fehlt, ist eine Analyse der Komplexität der Software-Entwicklung. Es wird kaum untersucht, wie hoch der Overhead oder die Einstiegshürden für Entwickler tatsächlich sind, wenn sie sich in abstrakte Konfigurationssprachen wie Kconfig und DeviceTrees einarbeiten müssen.

4 Forschungsfrage

Anstatt lediglich zu messen, wie schnell und effizient der finale Code auf dem Mikrocontroller ausgeführt wird, soll systematisch erarbeitet werden, welcher praktische Aufwand erforderlich ist, um grundlegende Funktionalitäten innerhalb des Zephyr-Ökosystems initial bereitzustellen. Aus der beschriebenen Problemstellung sowie dem aktuellen Stand der Technik leitet sich folgende zentrale Forschungsfrage ab:

Wie verhält sich der Overhead des Zephyr RTOS, insbesondere hinsichtlich des Konfigurationsaufwands sowie des Flash- und RAM-Bedarfs, zur tatsächlichen Applikations-Nutzlast bei einer einfachen Sensor-Edge-Anwendung?

5 Methodik

Um die Forschungsfrage nach dem Verhältnis von Konfigurationsaufwand zu funktionalem Mehrwert objektiv zu beantworten, wird ein iterativer Ansatz gewählt. Anstatt Zephyr nur theoretisch zu analysieren, wird ein funktionaler Prototyp („MoveLink“) entwickelt. Anhand der Entwicklung dieses Prototyps wird nach definierten Abschnitten überprüft, wie aufwendig die Implementierung, also das Schreiben von Code, im Vergleich zu der eigentlichen Konfiguration ist. Dieses Vorgehen erlaubt es, den Software-Overhead und die Lernkurve direkt während des Entstehungsprozesses zu messen. Das System „MoveLink“ besteht aus zwei Sensor-Knoten (Units), die Beschleunigungs- und Gyroskopdaten erfassen, und einer Zentraleinheit (Smartphone/PC), welche die Daten aggregiert (siehe Abbildung 1).

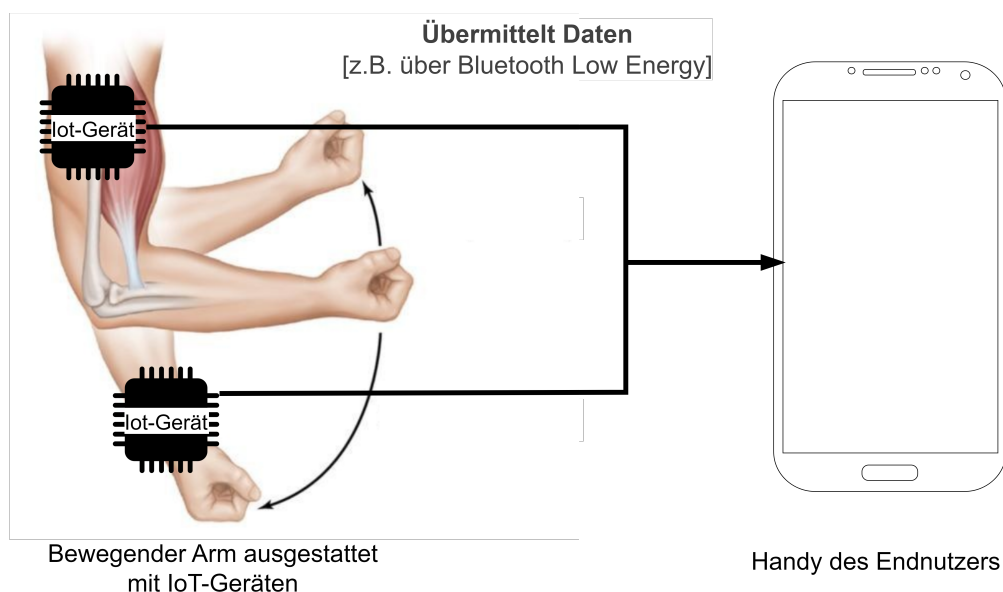


Abbildung 1: Abstrakter Aufbau von Movelink

Für das Projekt wird je ein Sensorknoten am Unter- und am Oberarm positioniert. Beide Einheiten erfassen kontinuierlich Bewegungsdaten und übertragen diese in Echtzeit an eine zentrale Instanz, beispielsweise ein Smartphone oder einen PC, welche die Daten aggregiert und weiterverarbeitet. Perspektivisch ließen sich diese Daten nutzen, um Bewegungsabläufe zu bewerten, wie etwa zur Überprüfung der korrekten Ausführung physiotherapeutischer Übungen. Wie bereits erläutert, liegt der Fokus hierbei jedoch strikt auf der Evaluierung von Zephyr als Plattform für verteilte Systeme und nicht auf der Gewinnung medizinisch verwertbarer Kennzahlen.

Die Entwicklung erfolgt nicht in einem Schritt, sondern in vier aufeinanderfolgenden Ausbaustufen (Meilensteinen). So lässt sich isolieren, welche Konfigurationsschritte (DeviceTree/Kconfig) spezifisch für welche Funktionalität anfallen.

Um den Entwicklungsaufwand quantifizierbar zu machen, wird der Prototyp schrittweise aufgebaut:

- **Baseline („Hello World“):**

Erfassung des minimalen Overheads, den Zephyr benötigt, um überhaupt auf der Hardware lauffähig zu sein (Board-Support-Package, Boot-Sequenz).

- **Hardware-Abstraktion:**

Messung des Aufwands, um externe Sensoren über den DeviceTree einzubinden. Hier wird analysiert, wie viel „Boilerplate-Code“ im Vergleich zur eigentlichen Datenverarbeitung nötig ist.

- **Konnektivität (1-zu-1-BLE):**

Evaluierung der Komplexität des Zephyr-Bluetooth-Stacks. Dies ist oft der Punkt, an dem der Flash- und RAM-Bedarf durch Protokoll-Overhead sprunghaft ansteigt.

- **1-to-N-Konnektivität (Datenübertragung im Netzwerk):**

Untersuchung der Skalierbarkeit. Hier wird geprüft, ob die Konfigurationskomplexität linear mit der Systemgröße wächst oder stagniert.

Bei jedem dieser Meilensteine wird erfasst, in welchem Verhältnis der Code-Overhead (Konfigurationsdateien, Setup) zum eigentlichen Applikationscode steht. Parallel dazu wird dokumentiert, wie sich der RAM- und Flash-Speicherbedarf mit jeder neuen Funktionsebene vergrößert. Diese gesteigerte Systemkomplexität liefert Erhebungswerte zur Beantwortung der Forschungsfrage.

6 Zeitplan

Um einen strukturierten Projektfortschritt zu gewährleisten, wurden den einzelnen Entwicklungsphasen konkrete Ziele und Artefakte zugeordnet. Diese Meilensteine bilden das Fundament der Evaluierung:

Phase 1: Baseline und Setup („Hello World“)

Ziel: Schaffung der infrastrukturellen Basis für die Implementierung sowie Erfassung des minimalen System-Overheads (Board-Support-Package, Boot-Sequenz).

Artefakt: Eine vollständig konfigurierte Toolchain (nRF Connect SDK) sowie ein lauffähiges Basisprojekt auf dem Zielsystem (XIAO nRF52840 Sense) zur initialen Messung des RAM- und Flash-Bedarfs.

Phase 2: Hardware-Abstraktion und Sensorik

Ziel: Ansteuerung der Hardware-Peripherie und Messung des Konfigurationsaufwands („Boilerplate-Code“ via DeviceTree) im Verhältnis zur eigentlichen Datenverarbeitung.

Artefakt: Eine Firmware, die auf den IMU-Sensor zugreift, die Rohdaten des Beschleunigungs- und Gyroskopsensors ausliest, filtert und seriell ausgibt. Dokumentation der aufgetretenen Barrieren.

Phase 3: 1-zu-1-Konnektivität (BLE)

Ziel: Etablierung einer drahtlosen Schnittstelle und Evaluierung der Komplexität sowie des sprunghaften Ressourcenbedarfs durch den Zephyr-Bluetooth-Stack.

Artefakt: Eine Firmware-Erweiterung, die die formatierten Sensordaten in Echtzeit drahtlos an ein Smartphone streamt, inklusive Erhebung der Metriken zum Protokoll-Overhead.

Phase 4: 1-to-N-Konnektivität und Skalierbarkeit

Ziel: Untersuchung der Systemskalierbarkeit, um zu prüfen, ob die Konfigurationskomplexität (DeviceTree/Kconfig) linear mit der Systemgröße wächst oder stagniert.

Artefakt: Ein funktionaler Verbund aus mindestens zwei Sensorknoten (Arm-System), die parallel und synchronisiert Daten an die Zentraleinheit senden.

Phase 5: Evaluation und Ausarbeitung

Ziel: Zusammenführung der phasenweise erhobenen Daten, wissenschaftliche Aufbereitung, Diskussion und schriftliche Reflexion der Forschungsfrage.

Artefakt: Die finale schriftliche Dokumentation der Projektarbeit inklusive eines detaillierten Mess- und Evaluationsprotokolls (Code-Ratio, Speicherbedarf) sowie dokumentiertem Quellcode (versioniert im Git-Repository).

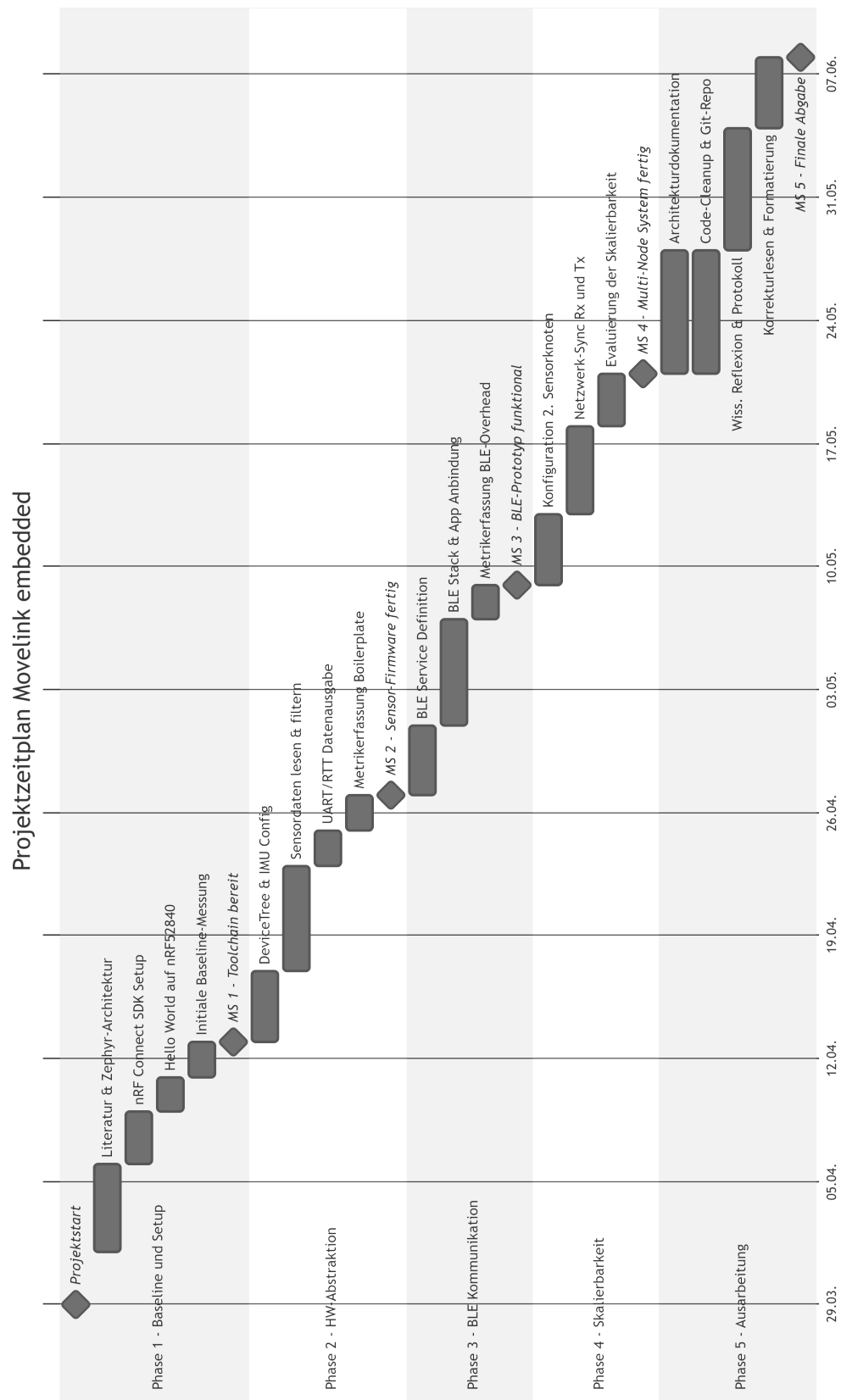


Abbildung 2: Zeitplan: Movelink embedded

Literaturverzeichnis

- [1] *Next generation smart ring integrates Nordic's nRF54L15 SoC delivering unparalleled processing and power consumption performance*, en. besucht am 1. Apr. 2026. Adresse: <https://www.nordicsemi.com/Nordic-news/2025/10/IDOs-IDR01-smart-ring-integrates-Nordics-nRF54L15-SoC>.
- [2] *Smart Home Monitoring With Nordic Thingy:91 & Edge Impulse*, en-US. besucht am 1. Apr. 2026. Adresse: <https://www.electromaker.io/project/view/smart-home-monitoring-with-edge-impulse>.
- [3] *Nordic NRF52840: Is Rust A Good Fit For Embedded Applications? – Punch Through*, en-US, Section: Embedded Software, Juni 2023. besucht am 2. Apr. 2026. Adresse: <https://punchthrough.com/nordic-nrf52840-is-rust-a-good-fit-for-embedded-applications/>.
- [4] (25) *Posten | LinkedIn*. besucht am 2. Apr. 2026. Adresse: https://www.linkedin.com/posts/jacobbeningo_zephyr-feels-complicated-good-because-activity-7338173311878688768-pyAr/.
- [5] M. Kanwal, M. Wajeeha und N. Khan, „IoT Devices Operating Systems Unveiled: An Analysis and Comparison of Operating System for Internet of Things,“ en, Jg. 14, Nr. 2, 2023.
- [6] M. Silva, D. Cerdeira, S. Pinto und T. Gomes, „Operating Systems for Internet of Things Low-End Devices: Analysis and Benchmarking,“ *IEEE Internet of Things Journal*, Jg. 6, Nr. 6, S. 10375–10383, Dez. 2019, ISSN: 2327-4662. DOI: 10.1109/JIOT.2019.2939008. besucht am 2. Apr. 2026. Adresse: <https://ieeexplore.ieee.org/document/8824106/>.
- [7] *Measuring Real-Time Operating System Performance – Part II: Comparing FreeRTOS vs. Zephyr*, en. besucht am 2. Apr. 2026. Adresse: <https://www.ul.com/sis/blog/measuring-real-time-operating-system-performance-part-ii-comparing-freertos-vs-zephyr>.